# Final Project Report: Online Payments Fraud Detection System

## 1. Executive Summary

In the modern digital economy, the exponential rise in online transactions has been accompanied by increasingly sophisticated financial fraud. This project presents an automated **Fraud Detection System** leveraging machine learning to identify suspicious activities in real-time. By utilizing a **Random Forest Classifier** integrated into a **Flask web application**, the system provides a scalable, accurate, and user-friendly solution to mitigate financial risk and enhance digital ecosystem security.

---

## 2. Problem Statement

Traditional fraud monitoring systems rely heavily on manual review and static, rule-based logic. These methods face three primary challenges:

- **Scalability:** Inability to process millions of transactions per second.
- **Adaptability:** Failure to recognize evolving fraud patterns.
- **Latency:** Manual intervention slows down transaction processing, leading to poor user experience.

---

## 3. System Architecture & Technical Stack

The system is built on a modular architecture designed for seamless data flow and low-latency predictions.

### 3.1 Technical Stack

| Layer | Technology Used |
|---|---|
| **Frontend** | HTML5, CSS3 (Responsive Design) |
| **Backend** | Flask (Python) |
| **Machine Learning** | Scikit-Learn, Random Forest, Pandas, NumPy |

| Layer | Technology Used |
| --- | --- |
| **Model Persistence** | Joblib (.pkl format) |

### 3.2 Workflow Pipeline

1. **User Input:** Transaction details are entered via a web-based form.
2. **API Processing:** Flask captures the data and converts it into a numeric format compatible with the model.
3. **Inference:** The pre-trained Random Forest model analyzes the features.
4. **Output:** The system returns a "Fraud" or "Legitimate" classification instantly.

---

## 4. Data Engineering & Model Development

### 4.1 Dataset Analysis

The model utilizes a transaction log dataset with features representing the financial state before and after a transaction. Key variables include:

- **Transaction Metadata:** `step` (temporal unit), `type` (TRANSFER, CASH_OUT, etc.).
- **Financial Metrics:** `amount`, `oldbalanceOrg`, `newbalanceOrig`, `oldbalanceDest`, and `newbalanceDest`.
- **Target:** `isFraud` (Binary classification).

### 4.2 Preprocessing Steps

- **Categorical Encoding:** One-hot encoding or label encoding applied to transaction `type`.
- **Data Cleaning:** Removal of redundant columns that do not contribute to predictive power (e.g., specific ID strings).
- **Data Splitting:** A standard **80/20 train-test split** was implemented to validate model performance on unseen data.

### 4.3 Algorithm Selection

While Decision Trees and Support Vector Machines (SVM) were evaluated, the **Random Forest Classifier** was selected for final deployment.

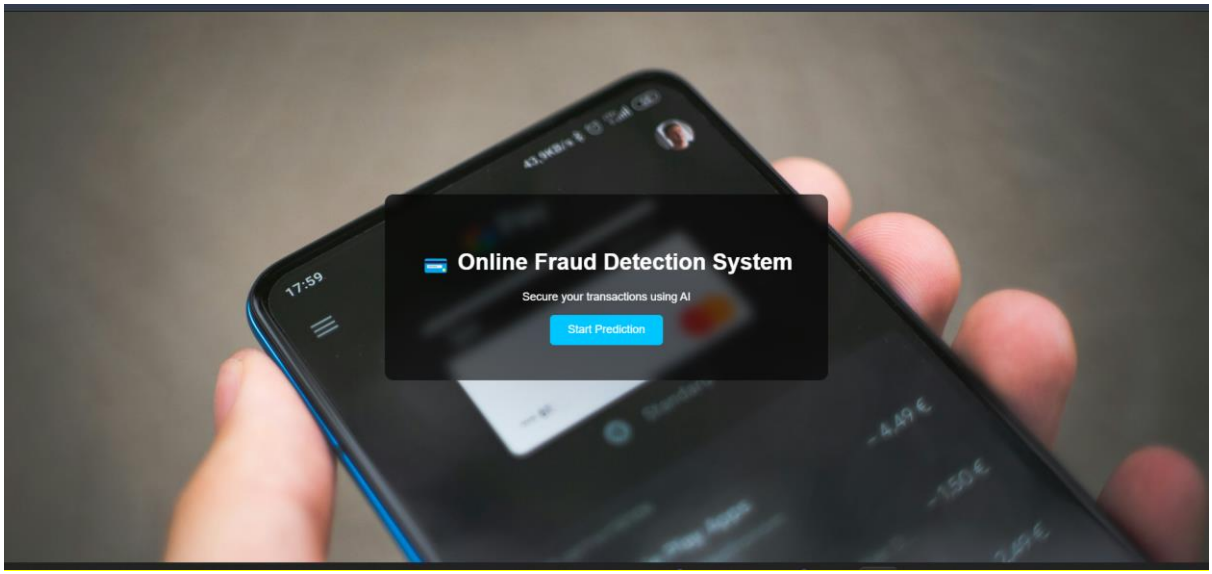
**Why Random Forest?** > It operates as an ensemble of decision trees, which significantly reduces the risk of **overfitting** (high variance) and handles the non-linear relationships common in fraudulent financial behaviour more effectively than single-model approaches.

---

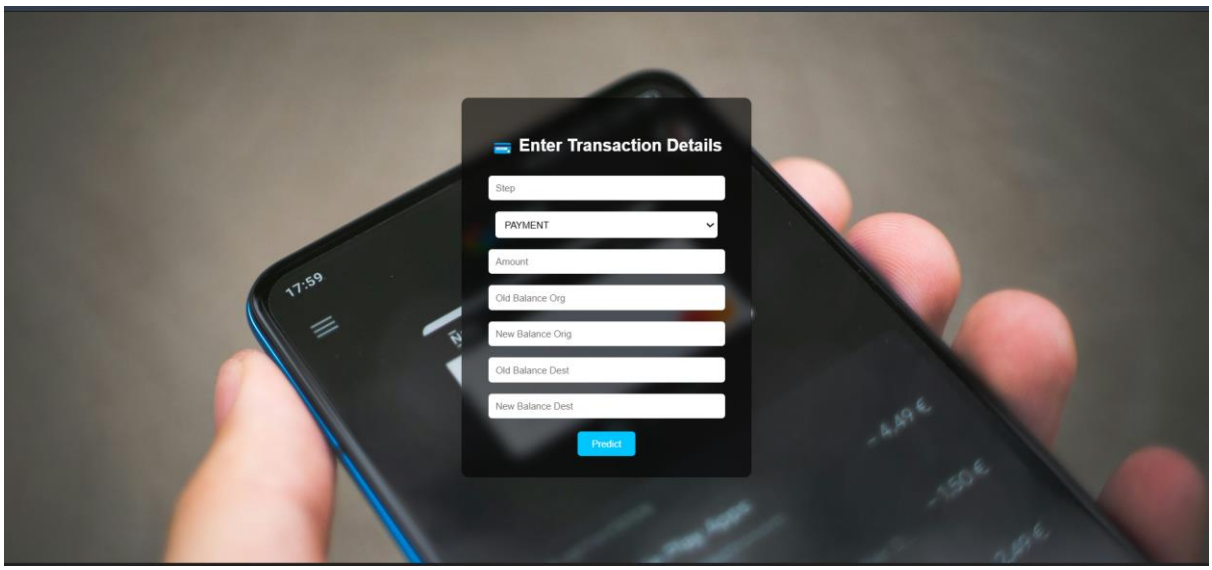
# 5. Web Interface & User Experience

The application provides a clean, three-tier interface:

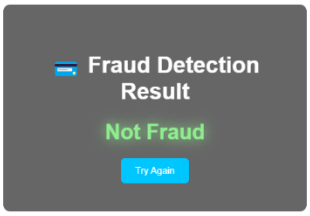
- **Home Dashboard:** Educates the user on the importance of fraud detection.




**Prediction Portal: A structured form requiring specific transaction inputs (Amount, Balances, etc.).**




- **Real-Time Results:** Visual feedback indicating the safety status of the transaction.




---

# 6. Performance Evaluation

The model was evaluated based on its ability to minimize two critical errors:

1. **False Positives:** Labelling a legitimate user as "Fraud" (affects user experience).
2. **False Negatives:** Missing an actual fraudulent transaction (affects security).

### Test Case Scenarios

| Scenario | Input Pattern | Result |
|---|---|---|
| **Standard Transaction** | Low amount, consistent balance | **Not Fraud** |
| **High-Risk Pattern** | Large transfer, immediate balance depletion | **Fraud Detected** |

---

# 7. Challenges & Future Scope

### Current Limitations

- **Static Data:** The model is trained on historical data; it requires periodic retraining to stay ahead of new fraud tactics.
- **Edge Cases:** Highly unusual but legitimate large-scale transfers may occasionally trigger false alarms.

### The Roadmap

- **Live Integration:** Connecting the Flask API directly to banking database streams for automated flagging.
- **Deep Learning:** Transitioning to Neural Networks (RNNs or LSTMs) to analyze the temporal sequences of a user's behavior.
- **Mobile Expansion:** Developing a cross-platform mobile app for administrators to monitor alerts on the go.

---

# 8. Conclusion

The **Online Payments Fraud Detection System** successfully bridges the gap between complex machine learning theory and practical application. By automating the detection process with a Random Forest backbone, the project provides a robust framework for reducing financial crime and fostering a safer digital payment environment.